Addressing Modes: Load and Store Instructions

Mastering Pre-Indexed, Post-Indexed, and Barrel-Shifter Scaling in ARM Cortex-M

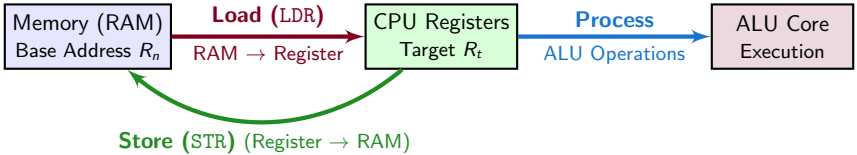
Microprocessor Systems

September 28, 2026

1. Broad Context: The Load-Store Architecture

Why Load and Store Instructions?

Unlike x86 CISC architectures, ARM uses a strict **Load-Store RISC Model**. ALU operations (addition, bitwise operations) **cannot directly manipulate data in RAM**. All processing occurs inside 32-bit internal registers.



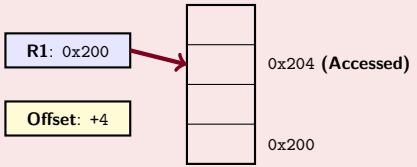
Bracket Notation

Brackets `[ ]` act as **pointer dereferencers**: `[R1]` means *"the memory location whose address is stored in R1"*.

2. Visual Intuition: Pre-Indexed vs. Post-Indexed

Pre-Indexed: Calculate Before Access

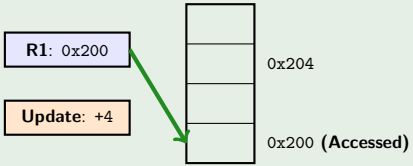
Offset is added to base **before** accessing memory.



Result: Accesses calculated address 0x204.

Post-Indexed: Access First, Update After

Memory accessed at base address **first**, then offset added.



Result: Accesses 0x200, then updates R1 ← 0x204.

3. Offset Application & Writeback Rules

Addressing Mode	Effective Memory Address	Base Register Update (R_n)
Simple Offset	Address $= R_n +$ offset	Unchanged (R_n)
Pre-Indexed (!)	Address $= R_n +$ offset	Updated ($R_n \leftarrow R_n +$ offset)
Post-Indexed	Address $= R_n$	Always Updated ($R_n \leftarrow R_n +$ offset)

The Writeback Indicator (!)

- The exclamation point ! explicitly commands auto-update (writeback) for pre-indexed access.
- Post-indexed access **implicitly updates** the base register without requiring a ! symbol.

4. Detailed Syntax: Immediate Offsets (#imm)

1. Simple Immediate Offset

```
LDR R0, [R1, #4]
@ Effective Address = R1 + 4
@ R1 remains unchanged
```

2. Pre-Indexed with Writeback

```
LDR R0, [R1, #4]!
@ Effective Address = R1 + 4
@ R1 auto-updates: R1 = R1 + 4
```

3. Post-Indexed (Implicit Writeback)

```
LDR R0, [R1], #4
@ Access Address = R1
@ After load: R1 = R1 + 4
```

Syntax Rule of Thumb

- Offset **inside** brackets → Pre-indexed
- Offset **outside** brackets → Post-indexed
- Trailing ! → Pre-indexed writeback

5. Dynamic Indexing: Register Offsets (R_m)

Why Register Offsets?

When array indices or offsets are computed dynamically at runtime, a secondary register R_m replaces the constant immediate `#imm`.

Simple Register Offset

```
LDR R0, [R1, R2]
@ Address = R1 + R2
@ R1 remains unchanged
```

Post-Indexed Register Offset

```
LDR R0, [R1], R2
@ Access Address = R1
@ R1 = R1 + R2
```

Pre-Indexed Register Offset

```
LDR R0, [R1, R2]!
@ Address = R1 + R2
@ R1 = R1 + R2
```

Use Case

Ideal for stepping through arrays where element stride is stored in a register (R_2).

6. Scaled Register Offsets & The Barrel Shifter

Hardware Barrel Shifter Integration

ARM cores integrate a hardware **Barrel Shifter** directly into the memory address generation unit. This allows multiplying an index register by 2^n in the **same single clock cycle** as memory access.

Assembly Syntax with Bitwise Shift

```
@ Address = R1 + (R2 * 4)
LDR R0, [R1, R2, LSL #2]

@ Pre-indexed with writeback
STR R0, [R1, R2, LSL #2]!

@ Post-indexed scaling
LDR R0, [R1], R2, LSL #2
```

C Data Type Scale Mapping

- LSL #1 ($\times 2$): 16-bit `short`
- LSL #2 ($\times 4$): 32-bit `int` / pointers
- LSL #3 ($\times 8$): 64-bit `double`

Efficiency: Eliminates explicit `MUL` instructions before array access!

7. Constant Loading & PC-Relative Literal Pools

Architectural Limitation

In 32-bit ARM instructions, immediate values are limited to 12 bits (8-bit value rotated). Arbitrary 32-bit memory addresses (e.g., 0x20000000) **cannot fit inside a standard MOV instruction**.

Method 1: Assembler Pseudo-Instruction

```
LDR R0, =0x20000000
LDR R1, =MY_DATA_LABEL
```

Assembler converts this into a **PC-relative load** from a near-by *Literal Pool*:

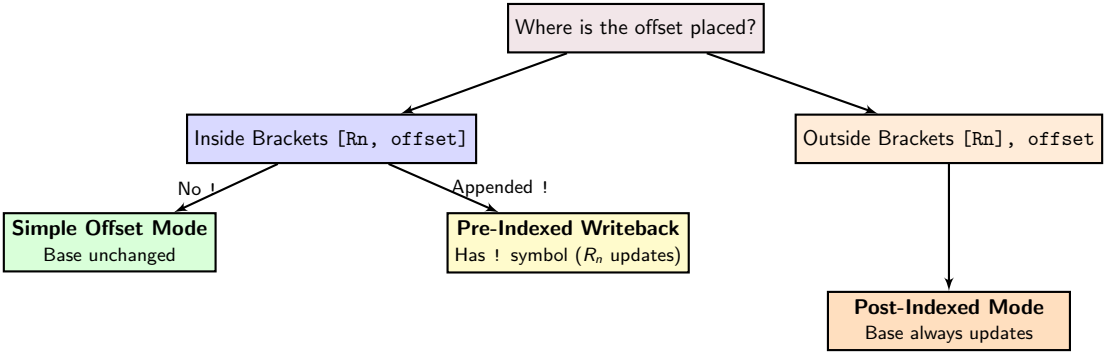
```
LDR R0, [PC, #offset]
```

Method 2: Two-Instruction Construct

```
@ Load lower 16 bits
MOVW R0, #0x0000
@ Load upper 16 bits
MOVT R0, #0x2000
```

Uses ARMv7-M `MOVW`/`MOVT` instructions to build a 32-bit constant directly into R_0.

8. Syntax Decision Tree & Summary



Quick Memory Rule

Pre-Indexed: Calculate address first → access memory.
Post-Indexed: Access memory first → update pointer.